

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
BỘ MÔN KHOA HỌC MÁY TÍNH

PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

CHƯƠNG 1

Một số khái niệm cơ bản

Phạm Hồng Phong

phongph@huce.edu.vn

Nội dung

- Giới thiệu môn học
- Một số khái niệm cơ bản
- Đánh giá độ phức tạp và chứng minh tính đúng

§ 1. Giới thiệu môn học

GIẢNG VIÊN

- TS. Phạm Hồng Phong
- Bộ môn KHMT, Khoa CNTT, Trường ĐH Xây dựng
- ĐT: 0983070744
- Email: phongph@huce.edu.vn

§ 1. Giới thiệu môn học

I. MỤC TIÊU MÔN HỌC

Kiến thức

- Trang bị kiến thức cơ bản về thiết kế và đánh giá thuật toán
- Cung cấp một số thuật toán cơ bản

Kỹ năng

- Hiểu được quá trình phân tích bài toán, xây dựng thuật toán, đánh giá độ phức tạp của thuật toán
- Triển khai chương trình thể hiện các thuật toán.

§ 1. Giới thiệu môn học

II. NỘI DUNG

- Các vấn đề về lý thuyết thuật toán
- Các phương pháp thiết kế thuật toán
- Các thuật toán thông dụng

§ 2. Một số khái niệm cơ bản

I. Bài toán (Problem)

- **Ví dụ 1.** Sắp xếp một dãy số

Cho: Một dãy số $a = (a_1, a_2, \dots, a_n)$

Cần: Xác định dãy $a' = (a'_1, a'_2, \dots, a'_n)$ là hoán vị của a sao cho $a'_1 \leq \dots \leq a'_n$

- **Ví dụ 2.** Tìm đường đi ngắn nhất từ u tới v của đồ thị $G=(V,E)$

Cho: Đồ thị G , và hai đỉnh u, v

Cần: xác định đường đi $d = (u=v_1, v_2, \dots, v_n=v)$ có độ dài ngắn nhất.

§ 2. Một số khái niệm cơ bản

- **Bài toán trong tin học:** lớp các bài toán cụ thể cùng loại
- Bài toán được cấu tạo bởi hai thành phần cơ bản:
 - Thông tin vào (input): Dữ liệu đã có
 - Thông tin ra (output): Yếu tố cần xác định
- ***Cho bài toán:*** cho input và output
- ***Giải bài toán:*** từ input, dùng một số hữu hạn các bước thao tác để tìm được output theo yêu cầu của bài toán

§ 2. Một số khái niệm cơ bản

II. Thuật toán (Algorithm)

Thuật toán là một dãy hữu hạn các thao tác đơn giản được sắp xếp theo một trình tự xác định dùng để giải một bài toán

- Một số đặc trưng của các thuật toán:
 - Tính tổng quát
 - Tính dừng
 - Tính xác định
 - Tính hiệu quả
- Một số phương pháp mô tả thuật toán:
 - Liệt kê từng bước
 - Sơ đồ khối
 - Giả mã (pseudo-code)

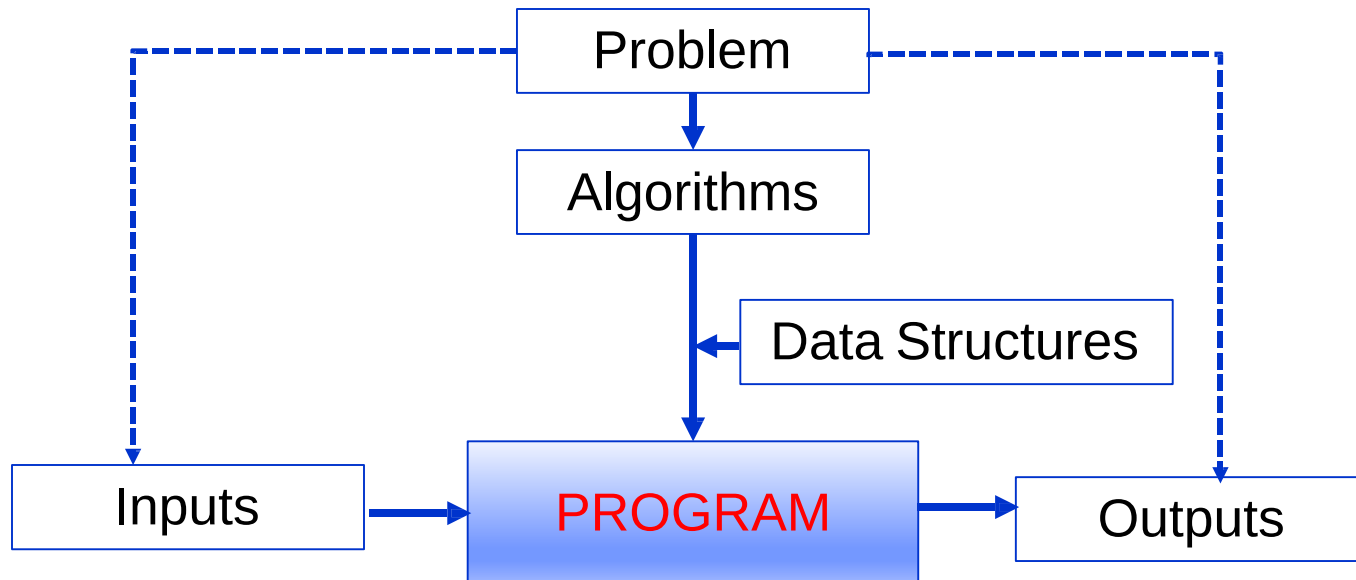
§ 2. Một số khái niệm cơ bản

- Nghiên cứu thuật toán quan tâm đến những vấn đề sau:
 - **Tính giải được:** Lớp bài toán nào giải được bằng thuật toán, lớp bài toán không giải được bằng thuật toán
 - **Tối ưu hóa thuật toán:** Tìm những thuật toán tốt hơn
 - **Triển khai thuật toán:** Sử dụng các ngôn ngữ lập trình để thực hiện thuật toán trên máy tính
- Hai lĩnh vực nghiên cứu chính
 - Thiết kế và đánh giá thuật toán
 - Triển khai thuật toán (lập trình)

§ 2. Một số khái niệm cơ bản

III. Chương trình (PROGRAM)

- Chương trình là sự thể hiện bằng một ngôn ngữ lập trình cụ thể một thuật toán đã cho được xác định trên một cấu trúc dữ liệu xác định
- Sơ đồ giải bài toán trên máy tính



§ 3.Đánh giá độ phức tạp và chứng minh tính đúng

I. Đánh giá độ phức tạp

- **Phương pháp thực nghiệm:** Lập trình và thử trên các ví dụ
- **Phương pháp lý thuyết:** Tính toán thời gian, bộ nhớ, ... cần thiết của mỗi thuật toán dựa theo độ lớn của input

Ưu điểm:

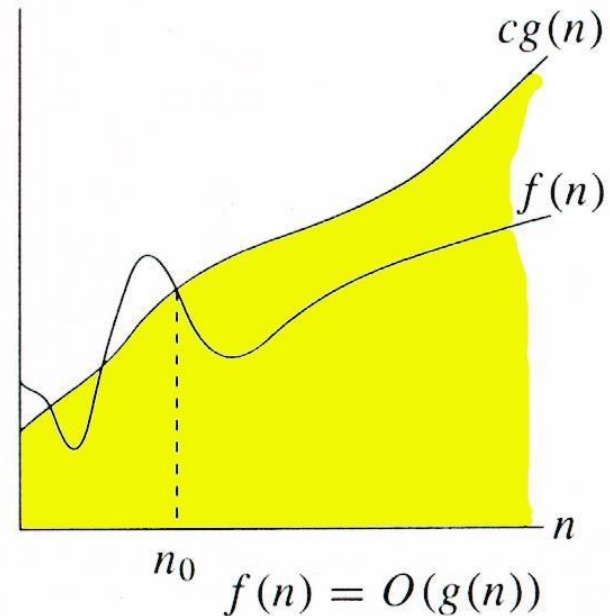
- ✓ Không phụ thuộc ngôn ngữ lập trình, loại máy tính
- ✓ Biết được tính hiệu quả của thuật toán đối với các dữ liệu có kích thước lớn

§ 3.Đánh giá độ phức tạp và chứng minh tính đúng

Ký hiệu O (đọc là Ô lớn)

$$f(n) = O(g(n)) \Leftrightarrow \exists c, n_0 \text{ sao cho } \forall n \geq n_0 : f(n) \leq c \cdot g(n)$$

Ví dụ: Xét $T(n) = n^2 + n$
Với mọi $n \geq 1$: $T(n) \leq 2n^2$
Suy ra $T(n) = O(n^2)$



§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Nhận xét: Nếu $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = K$ với $K < +\infty$ thì $f(n) = O(g(n))$

Chứng minh: Theo đn giới hạn của dãy số:

$$\exists n_0 : \left| \frac{f(n)}{g(n)} - K \right| < 1 \quad \text{với mọi } n \geq n_0$$

$$\text{Lại có } \left| \frac{f(n)}{g(n)} - K \right| \geq \frac{f(n)}{g(n)} - K$$

$$\text{Suy ra } f(n) \leq (K + 1)g(n) \quad \text{với mọi } n \geq n_0 \text{ (đpcm)}$$

§ 3. Đánh giá độ phức tạp và chứng minh tính

Quy tắc đánh giá độ phức tạp thuật toán

1. Quy tắc bỏ hằng số

Nếu $T(n) = O(c_1 \cdot f(n))$ thì $T(n) = O(f(n))$

Chứng minh:

$T(n) = O(c_1 \cdot f(n)) \Rightarrow T(n) \leq c c_1 \cdot f(n)$ với mọi $n \geq n_0$

Chọn $c' = c c_1$ suy ra

$T(n) \leq c' \cdot f(n)$ với mọi $n \geq n_0$

Do đó $T(n) = O(f(n))$

Ví dụ: $T(n) = O(2n^2) \Rightarrow T(n) = O(n^2)$

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

2. Quy tắc cộng

Nếu thuật toán P có hai thành phần liên tiếp P_1 và P_2 với độ phức tạp là $T_1(n) = O(f(n))$ và $T_2(n) = O(g(n))$ thì P có độ phức tạp là $T(n) = O(f(n) + g(n))$.

Chứng minh:

$$T_1(n) \leq c_1 \cdot f(n) \text{ với mọi } n \geq n_1$$

$$T_2(n) \leq c_2 \cdot g(n) \text{ với mọi } n \geq n_2$$

Chọn $c = \max(c_1, c_2)$; $n_0 = \max(n_1, n_2)$, ta có:

$$T(n) \leq c(f(n) + g(n)) \text{ với mọi } n \geq n_0 \quad (\text{đpcm})$$

$$\text{Ví dụ: } T_1(n) = O(n); T_2(n) = O(n^2) \Rightarrow T(n) = O(n + n^2)$$

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

3. Quy tắc lấy max

$T(n) = O(f(n)+g(n))$ và $\lim[f(n):g(n)] = 0 \Rightarrow T(n) = O(g(n))$.

Chứng minh:

- $T(n) = O(f(n)+g(n)) \Rightarrow T(n) \leq c (f(n)+g(n))$ với mọi $n \geq n_1$
- $\lim[f(n):g(n)] = 0 \Rightarrow f(n) \leq g(n)$ với mọi $n \geq n_2$

Chọn $n_0 = \max(n_1 ; n_2)$:

$$T(n) \leq c (g(n)+g(n)) = 2c \cdot g(n) \text{ với mọi } n \geq n_0$$

Chọn $c'=2c$:

$$T(n) \leq c' \cdot g(n) \Rightarrow \text{Đpcm}$$

Ví dụ: $T(n) = n^4 + 3n^2 + 1 \Rightarrow T(n) = O(n^4)$

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

4. Quy tắc nhân

Nếu thuật toán P được thực hiện bằng cách thực hiện n lần thuật toán P^* .

Giả sử P^* có độ phức tạp là $T_{P^*}(n) = O(f(n))$ thì P có độ phức tạp là $T_P(n) = O(n \cdot f(n))$.

Chứng minh:

$$T_{P^*}(n) \leq c \cdot f(n) \text{ với mọi } n \geq n_0$$

$$\Rightarrow T_P(n) = n \cdot T_{P^*}(n) \leq c \cdot n \cdot f(n) \text{ với mọi } n \geq n_0$$

$$\Rightarrow T_P(n) = O(n \cdot f(n)) \text{ (đpcm)}$$

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Phân tích thời gian thực hiện thuật toán *Sắp xếp chèn*

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

=> Thuật toán có độ phức tạp thời gian là $O(n^2)$

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

II. Chứng minh tính đúng của thuật toán

Các chiến lược chứng minh tính đúng (correctness)

- **Kiểm thử (testing):** Chạy thử thuật toán với các dữ liệu vào cụ thể
Ưu điểm: Dễ thực hiện
Nhược điểm: Có thể không phát hiện hết các lỗi
- **Chứng minh tính đúng (correctness proof):** chứng minh bằng toán học
Ưu điểm: Tổng quát
Nhược điểm: Khó hơn và có thể vẫn có lỗi
- **Kết hợp kiểm thử và chứng minh tính đúng**

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Các phương pháp chứng minh tính đúng

- ***Đối với thuật toán đệ quy:*** Dùng quy nạp
- ***Đối với thuật toán không đệ quy:*** sử dụng bất biến vòng lặp (loop invariant)

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Chứng minh tính đúng đối với thuật toán đệ quy

Phương pháp quy nạp: Chứng minh tính đúng của thuật toán theo kích thước dữ liệu vào (n)

- Cơ sở của quy nạp: Chứng minh thuật toán đúng trong trường hợp đơn giản nhất của n .
- Quy nạp: Giả sử thuật toán đúng với $n=k$. Chứng minh thuật toán đúng với $n = k+1$



§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Ví dụ: Thuật toán tìm max của một dãy số $A_1, A_2, \dots, A_n$ có n phần tử

```
Maximum( $n$ )  $\equiv$  // Tìm max của dãy số có  $n$  phần tử  
    if ( $n==1$ ) return ( $A_1$ )  
    else return (max(Maximum( $n-1$ ),  $A_n$ )) ;  
End.
```

Chứng minh tính đúng

- $n=1$: Maximum(1) trả về $A_1 \Rightarrow$ Thuật toán đúng với $n=1$.
- Giả sử thuật toán đúng với $n=k$, tức là Maximum(k) trả về $\max(A_1, \dots, A_k)$. Khi đó: Maximum($k+1$) trả về $\max(\text{Maximum}(k), A_{k+1}) = \max(A_1, \dots, A_k, A_{k+1})$
 $\Rightarrow$ ĐPCM

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Chứng minh tính đúng đối với thuật toán không đệ quy

Sử dụng bất biến vòng lặp (*loop invariant*)

Bất biến vòng lặp (*loop invariant*)

- Bất biến vòng lặp là một khẳng định liên quan đến ***các lần lặp***
- Bất biến đúng trong lần lặp cuối cùng kéo theo thuật toán đúng

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Chứng minh tính đúng của thuật toán bằng cách sử dụng bất biến vòng lặp:

- Xác định bất biến vòng lặp sao cho: bất biến vòng lặp đúng ở ***lần lặp cuối cùng*** kéo theo thuật toán đúng
- Chứng minh bất biến đúng ở lần lặp đầu tiên
- Chứng minh bất biến đúng ở lần lặp trước thì cũng đúng ở lần lặp sau

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Ví dụ: Thuật toán tìm max của một dãy số $A_1, A_2, \dots, A_n$ có n phần tử

Maximum (n) $\equiv$ // Tìm max của dãy số có n phần tử

m=A1;

for (i=2; i<=n; i++)

if (m < Ai) m=Ai;

return (m)

End.

Bất biến vòng lặp:

với mọi $j=2, 3, \dots, n$ ta có:

Sau lần lặp ứng với $i = j$ ta có: $m = \max(A_1, \dots, A_j)$ (*)

§ 3. Đánh giá độ phức tạp và chứng minh tính đúng

Ta chứng minh bất biến vòng lặp bằng quy nạp

- Sau lần lặp với $i = 2$: $m = \max(m, A_2) = \max(A_1, A_2) \Rightarrow (*)$ đúng với $i = 2$
- Giả sử sau lần lặp ứng với $i = j$ ta có $m = \max(A_1, \dots, A_j)$
 $\Rightarrow$ sau lần lặp ứng với $i = j+1$ ta có:
$$m = \max(m, A_{j+1}) = \max(\max(A_1, \dots, A_j), A_{j+1}) = \max(A_1, \dots, A_j, A_{j+1})$$

Như vậy, $(*)$ đúng với mọi $i = 2, 3, \dots, n$.

Nói riêng: $(*)$ đúng với $i = n$ và do đó, sau khi thoát khỏi vòng lặp ta có:

$$m = \max(A_1, A_2, \dots, A_n) \text{ (đpcm)}$$

Bài tập 1

Xây dựng thuật toán tìm số Fibonacci thứ n với độ phức tạp $O(n)$

Bài tập 2

Cài đặt thuật toán giải bài toán Tháp Hà Nội

Bài tập 3

Cho một dãy gồm n số nguyên dương, xác định xem có tồn tại dãy con có tổng bằng k hay không?

- a) Đưa ra thuật toán có thời gian thực hiện $O(n^3)$
- b) Đưa ra thuật toán có thời gian thực hiện $O(n^2)$
- c) Đưa ra thuật toán có thời gian thực hiện $O(n)$
- d) Thực nghiệm các thuật toán đã đưa ra

Bài tập 4

Cho A là một mảng gồm n phần tử. Thuật toán sau đây cho phép tìm khoảng cách giữa hai số gần nhau nhất của mảng:

```
Mindistance(A) {  
    dmin =  $+\infty$ ;  
    for (i=0; i≤n-1)  
        for (j=0; j≤n-1)  
            if (i≠j and |A[i]-A[j]|<dmin  
                dmin = |A[i]-A[j]|;  
}
```

- a) Đánh giá thời gian thực hiện của thuật toán trên.
- b) Cải tiến thuật toán.
- c) Thực nghiệm.

Bài tập 5

Cho một dãy gồm n số, tìm dãy con liên tiếp có tổng lớn nhất.

- a) Đưa ra thuật toán có thời gian thực hiện $O(n^3)$
- b) Đưa ra thuật toán có thời gian thực hiện $O(n^2)$
- c) Đưa ra thuật toán có thời gian thực hiện $O(n \log n)$
- d) Thực nghiệm các thuật toán đã đưa ra